

Understanding Pipelining Hazards

BLG 483E – Artificial Intelligence

salihsefer36

May 10, 2026

Introduction to CPU Pipelining

Think of a restaurant: one waiter takes orders, one chef cooks, another plates, another serves. If one person did everything per customer, the restaurant would serve one table at a time. A CPU pipeline works exactly the same way.

- A pipeline splits every instruction into 5 stages that run **simultaneously** for different instructions: **IF** (fetch), **ID** (decode), **EX** (execute), **MEM** (memory), **WB** (write result).
- When the assembly line runs smoothly, the CPU completes roughly one instruction per clock cycle (instead of one every 5 cycles).
- A **hazard** is anything that breaks the smooth flow — a conflict that forces a stage to wait or throw away work.
- Hazards come in three flavours: **data**, **structural**, and **control**.

Data Hazards — Types

Imagine a relay race where runner B must grab the baton from runner A before sprinting. If B takes off before A arrives, the handoff fails. Data hazards are exactly that: one instruction needs a value the previous one has not finished computing yet.

- **RAW** (Read-After-Write, “true dependence”): the most common type. Instruction B tries to read a register before instruction A has written its result into it. *Runner B starts before the baton arrives.*
- **WAR** (Write-After-Read, “anti-dependence”): B wants to overwrite a register that A is still reading. *Chef B wants to clean the pan that chef A is still using.*
- **WAW** (Write-After-Write, “output dependence”): two instructions both write the same register; the second write must “win”. *Two workers submit the same report; only the later version should be filed.*

Data Hazards — Detection & Mitigation

Think of a traffic officer watching an intersection. When two cars are about to collide, the officer either holds one back (stall) or waves it through a special shortcut lane (forwarding).

- **Detection:** A hazard detection unit (“the traffic officer”) checks the pipeline registers (ID/EX, EX/MEM, MEM/WB) every cycle to spot conflicts before they cause wrong results.
- **Stall (interlock):** Freeze the fetch and decode stages, insert a bubble (an empty “do nothing” slot) — like holding a car at a red light for one cycle.
- **Forwarding (bypassing):** Route the computed result directly from one pipeline stage to the next instruction’s input, skipping the detour through the register file. Eliminates most RAW stalls completely.
- **Load-use exception:** When instruction A loads from memory and B immediately needs that value, forwarding still cannot help — one stall cycle is unavoidable.

Structural Hazards

Two delivery trucks arriving at a warehouse with a single loading dock at the same time: one must wait. Structural hazards are the same idea — two pipeline stages need the same piece of hardware simultaneously.

- **Memory conflict:** If the CPU has only one memory unit (no separate instruction cache and data cache), the Fetch stage and the Memory stage cannot both run in the same cycle — one must stall.
- **Register write-port conflict:** A register file with a single write port cannot accept two write-back results in the same cycle. *One printer, two print jobs queued at the same instant.*
- **Multi-cycle unit contention:** Slow operations (floating-point divide, square root) tie up a shared unit for many cycles, blocking other instructions from using it.
- **Fixes:** Add duplicate hardware (two memory ports), share the resource across alternating cycles, or let the compiler schedule independent work in between.

Control Hazards

*You are driving and your GPS says “turn left” — but you are already in the right lane three blocks back. You have “fetched” the wrong road. A branch hazard is the same: the CPU grabs the wrong next instructions before it knows which way a branch (an *if/loop*) goes.*

- A branch's direction and destination are only known at the EX stage, so 2 instructions are already fetched for the wrong path — they must be **flushed** (thrown away), wasting 2 cycles.
- **Static prediction:** Always guess “branch not taken” (keep driving straight). Simple, but wrong for loops.
- **Dynamic prediction (2-bit counter):** The CPU remembers past behaviour in a small table and guesses based on history — like a GPS that learns your usual route.
- **Branch Target Buffer (BTB):** Caches the destination address so the CPU can jump immediately without recomputing where to go.
- **Delayed branch:** The slot right after a branch always executes; the compiler fills it with a useful instruction instead of wasting it.

Hazard Mitigation Summary

A factory supervisor has three tools: a stop button (stall), a conveyor shortcut (forwarding), and a smart pre-planned schedule (compiler reordering). Together they keep the assembly line moving even when hiccups occur.

- **Stall:** Pause the front of the pipeline and insert an empty “bubble” instruction — buys time for the slow stage to finish, but costs cycles.
- **Forwarding:** A hardware shortcut that hands a result directly from one stage to the next instruction’s input, cutting most RAW stalls to zero extra cycles.
- **Load-use stall:** The one case forwarding cannot fully fix; a single wasted cycle is unavoidable unless the compiler reorders code.
- **Branch delay slot:** The instruction after a branch always runs; filling it with real work (not a NOP) recovers a cycle for free.
- **Compiler scheduling:** By rearranging independent instructions at compile time, the compiler can eliminate many hazards before the hardware ever sees them.

References



Patterson, D. A. and Hennessy, J. L. *Computer Organization and Design: The Hardware/Software Interface*, 5th ed. Morgan Kaufmann, 2014.



Hennessy, J. L. and Patterson, D. A. *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2017.